

Running Sum of 1D Array:-

Code:-

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for(int i=1;i<nums.length;i++){  
            nums[i]=nums[i]+nums[i-1];  
        }  
        return nums;  
    }  
}
```

Output:-

The screenshot displays the LeetCode problem page for 'Running Sum of 1D Array'. The code is written in Java and is accepted. The test results show the input [1, 2, 3, 4] resulting in the output [1, 3, 6, 10].

Runtime: 0 ms | Beats 100.00%

Memory: 44.34 MB | Beats 35.20%

Testcase Results:

Case	Input	Output	Expected
Case 1	nums = [1, 2, 3, 4]	[1, 3, 6, 10]	[1, 3, 6, 10]

Shuffle the Array:-

Code:-

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] result = new int[2 * n];  
        for (int i = 0; i < n; i++) {  
            result[2 * i] = nums[i];  
            result[2 * i + 1] = nums[i + n];  
        }  
        return result;  
    }  
}
```

Output:-

The screenshot displays the LeetCode submission interface for the 'Shuffle the Array' problem. The code is written in Java and is shown in the 'Code' tab. The runtime performance is 0 ms, beating 100.00% of other submissions. The memory usage is 46.55 MB, beating 41.21% of other submissions. The test results show that the solution is 'Accepted' and passes all three test cases (Case 1, Case 2, Case 3). The input for the test cases is nums = [2, 5, 1, 3, 4, 7] and n = 3, and the output is [2, 3, 5, 4, 1, 7].

Code:

```
public int[] shuffle(int[] nums, int n) {  
    int[] result = new int[2 * n];  
    for (int i = 0; i < n; i++) {  
        result[2 * i] = nums[i];  
        result[2 * i + 1] = nums[i + n];  
    }  
    return result;  
}
```

Runtime: 0 ms | Beats 100.00%

Memory: 46.55 MB | Beats 41.21%

Test Results: Accepted (Runtime: 0 ms)

Case 1: Input: nums = [2, 5, 1, 3, 4, 7], n = 3. Output: [2, 3, 5, 4, 1, 7].